
Kubernetes CronJobs

Lab Overview

This document teaches **time-based task scheduling** in:

1. **Linux (Cron / crontab)**
2. **Kubernetes (CronJob resource)**

You will:

- Understand how cron works at the OS level
 - Learn crontab syntax deeply
 - Map Linux cron concepts to Kubernetes CronJobs
 - Create, observe, troubleshoot, and manage Kubernetes CronJobs
 - Understand real-world use cases and best practices
-

Learning Objectives

By the end of this lab, you will be able to:

- Explain how the cron daemon works
- Write and interpret crontab schedules
- Use special schedule strings
- Create and manage Linux cron jobs
- Understand Kubernetes CronJob architecture
- Create, monitor, and debug Kubernetes CronJobs
- Explain differences between Linux cron and Kubernetes CronJobs

PART 1: Cron Jobs in Linux

What Is a Cron Job?

A **cron job** is a **scheduled command or script** that runs automatically at a specified time or interval.

It is powered by a background service called **cron daemon (crond)**.

Cron Daemon (**crond**)

Key Characteristics

- Runs continuously in the background
- Wakes up **every minute**
- Checks all crontab files
- Executes commands whose schedule matches the current time

 If **crond** is stopped → **no cron jobs will run**

Check status:

```
systemctl status crond
```

What Is a Crontab?

A **crontab (cron table)** is a configuration file that defines scheduled jobs.

Types of Crontabs

Type	Location
User crontab	Managed via <code>crontab -e</code>
System crontab	<code>/etc/crontab</code>
Periodic jobs	<code>/etc/cron.daily</code> , <code>/etc/cron.hourly</code> , etc

Crontab Commands

View crontab

`crontab -l`

Edit crontab

`crontab -e`

Remove all cron jobs

`crontab -r`

 **Dangerous** – removes all scheduled jobs for the user.

Crontab Syntax (Deep Dive)

* * * * * command_to_run

```

| | | | |
| | | | └─ Day of week (0–7) → Sunday = 0 or 7
| | | └─── Month (1–12)
| | └───── Day of month (1–31)
| └──────── Hour (0–23)
└────────── Minute (0–59)

```

Special Characters

Symbo	Meaning
-------	---------

*	Every value
---	-------------

,	Multiple values
---	-----------------

-	Range
---	-------

/	Step value
---	------------

Examples

`* / 5 * * * *` # Every 5 minutes

`0 9-17 * * *` # Every hour from 9am–5pm

`0 0 1 * *` # First day of every month

Example Cron Job

`0 2 * * * /home/user/backup.sh`

✓ Runs every day at **2:00 AM**

Special Schedule Strings

String	Description
--------	-------------

@reboo	Run at startup
--------	----------------

t	
---	--

@hourly Every hour

@daily Once per day

@weekly Once per week

@monthly Once per month

@yearly Once per year

Example:

@daily /home/user/clean_temp.sh

Common Linux Cron Use Cases

- Automated backups
 - Log rotation
 - Disk cleanup
 - Database dumps
 - Health-check scripts
 - Sending periodic reports
-

PART 2: Kubernetes CronJobs

What Is a Kubernetes CronJob?

A **Kubernetes CronJob** is the **Kubernetes-native equivalent of a Linux cron job**.

📌 Instead of running commands on a server:

- Kubernetes schedules **Jobs**
 - Jobs create **Pods**
 - Pods run containers at scheduled times
-

🧠 Architecture of a Kubernetes CronJob

CronJob

↓ (on schedule)

Job

↓

Pod

↓

Container runs task

➡ After execution:

- Pod exits
 - Job completes
 - Kubernetes keeps history (configurable)
-

🔄 Linux Cron vs Kubernetes CronJob

Linux Cron	Kubernetes CronJob
Runs on OS	Runs in cluster
Executes commands	Runs containers
Single machine	Distributed system

No retries by default	Retry & failure handling
Manual logging	Logs via kubectl

Lab: Create a Kubernetes CronJob

Step 1: Create CronJob YAML

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.28
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo "Hello from the Kubernetes cluster"
          restartPolicy: OnFailure
```

Step 2: Apply the CronJob

```
kubectl apply -f cronjob.yaml
```

Step 3: Verify CronJob

```
kubectl get cronjob
```

You should see:

```
SCHEDULE    SUSPEND    ACTIVE    LAST SCHEDULE
* * * * *   False     0         <timestamp>
```

Step 4: Observe Jobs Created

```
kubectl get jobs
```

Every minute:

- A new **Job** appears
 - Each Job creates a Pod
-

Step 5: View Pods

```
kubectl get pods
```

Completed pods will show:

```
STATUS: Completed
```

Step 6: View Logs

kubectl logs <pod-name>

Expected output:

Mon Feb 3 13:25:00 UTC 2026

Hello from the Kubernetes cluster

CronJob Configuration Options

Concurrency Policy

concurrencyPolicy: Forbid

Policy	Behavior
--------	----------

Allow	Default (overlap allowed)
-------	---------------------------

Forbid	Skip if previous is running
--------	-----------------------------

Replace	Kill previous job
---------	-------------------

Job History Limits

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

Controls cleanup of old Jobs.

Suspend a CronJob

```
kubecttl patch cronjob hello -p '{"spec":{"suspend":true}}'
```

Delete CronJob

```
kubecttl delete cronjob hello
```

Common Mistakes & Gotchas

1. **Using local paths**
 - Pods are ephemeral
 2. **Too frequent schedules**
 - Causes Job pile-up
 3. **No job history limits**
 - Cluster clutter
 4. **Long-running jobs**
 - Should be Deployments, not CronJobs
-
-

Production Use Cases for Kubernetes CronJobs

Database Backups

Why: Protect data against loss or corruption.

Examples:

- Backup MySQL every night at 2 AM:

schedule: "0 2 * * *"

command:

- /bin/sh

- -c

- mysqldump -u root -p\$MYSQL_PASSWORD mydb > /backup/mydb_\$(date +%F).sql

- Backup MongoDB every 6 hours
- Export Postgres DB snapshots to S3

Notes:

- CronJob pods can mount **Persistent Volumes** or use **cloud storage**.
 - You can set `successfulJobsHistoryLimit` to avoid storing too many backups in the cluster.
-

2 Log Rotation & Cleanup

Why: Prevent disk space from filling up, maintain system hygiene.

Examples:

- Delete logs older than 7 days:

schedule: "0 0 * * *"

command:

- /bin/sh

- -c

- find /var/log/app -type f -mtime +7 -delete

- Rotate log files and compress them
 - Archive logs to S3 for auditing
-

3 ETL (Extract, Transform, Load) Jobs

Why: Automate data pipelines for analytics.

Examples:

- Extract sales data every hour from DB → transform → upload to data warehouse
- Daily aggregation of metrics for dashboards
- Batch processing of CSV files in object storage

Notes:

- CronJobs are ideal for **short-lived, stateless batch tasks**.
 - For long-running pipelines, consider **Jobs with queue systems**.
-

4 Email Reports and Notifications

Why: Send periodic alerts, dashboards, or reports automatically.

Examples:

- Daily sales report at 8 AM:

schedule: "0 8 * * *"

command:

- /bin/sh
- -c
- python /scripts/send_daily_report.py

- Weekly system health email
 - Monthly billing invoices generation
-

5 Cache Warming / Maintenance

Why: Keep systems responsive or optimized.

Examples:

- Prepopulate Redis cache every morning before peak traffic
 - Rebuild search indexes overnight
 - Periodic cleanup of temporary files
-

6 Security & Compliance Tasks

Why: Automate critical security checks.

Examples:

- Run vulnerability scans weekly
 - Rotate encryption keys or secrets periodically
 - Verify configuration compliance with policies
-

7 Monitoring & Health Checks

Why: Ensure system reliability.

Examples:

- Check microservices API endpoints every 5 minutes
 - Run scripts that validate cluster resources
 - Trigger alerts if a pod/service is unhealthy
-

Data Synchronization

Why: Sync data between systems without manual intervention.

Examples:

- Sync S3 → GCS buckets every night
 - Copy database snapshots to DR (Disaster Recovery) region
 - Sync local filesystem with cloud storage
-

Best Practices in Production

Practice	Reason
Limit <code>concurrencyPolicy</code>	Avoid overlapping jobs
Set <code>successfulJobsHistoryLimit</code>	Prevent resource overload
Use <code>restartPolicy: OnFailure</code>	Retry failed jobs automatically

Use **service accounts with minimal permissions**

Security principle of least privilege

Use **ConfigMaps/Secrets**

Keep scripts and credentials out of images

Monitor Job logs & metrics

Ensure jobs are executing as expected

Key Takeaways

- **CronJobs are for scheduled, repeatable tasks**
 - Ideal for **batch, maintenance, backup, and reporting workloads**
 - Use **Kubernetes CronJobs** instead of OS cron in cloud-native environments for **resilience, scalability, and observability**
-